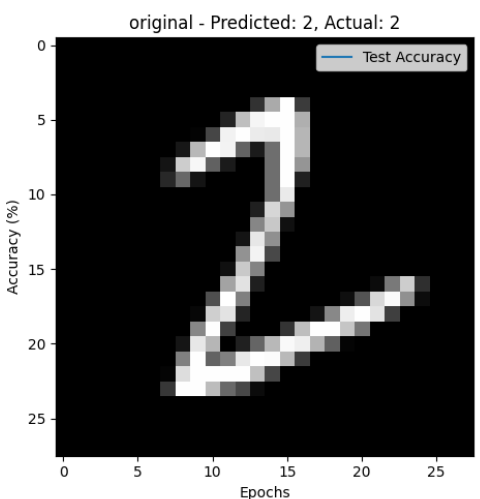
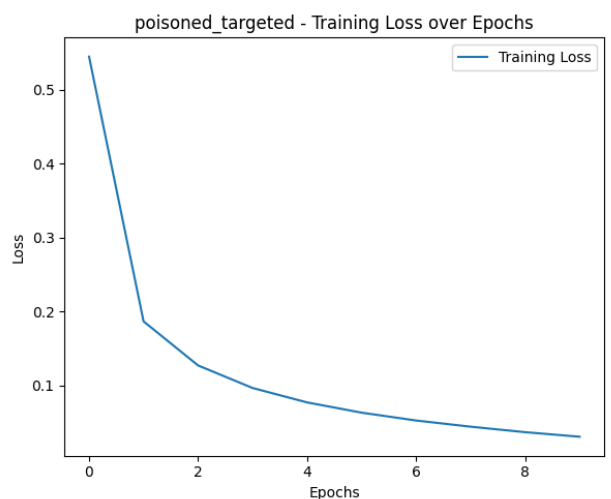
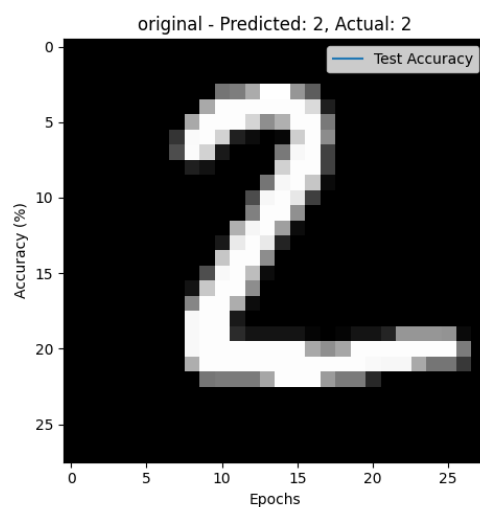
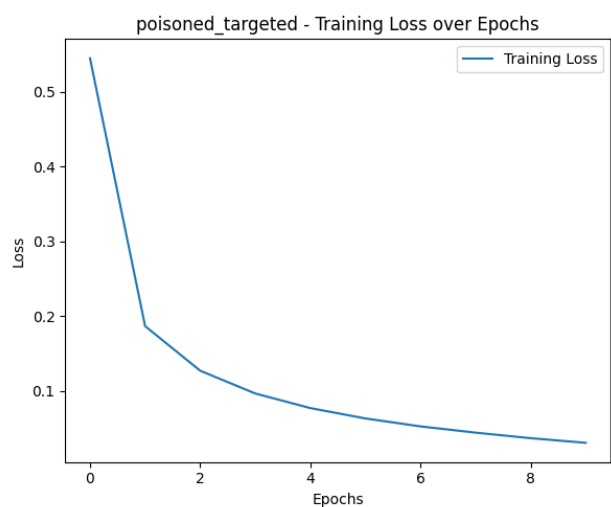
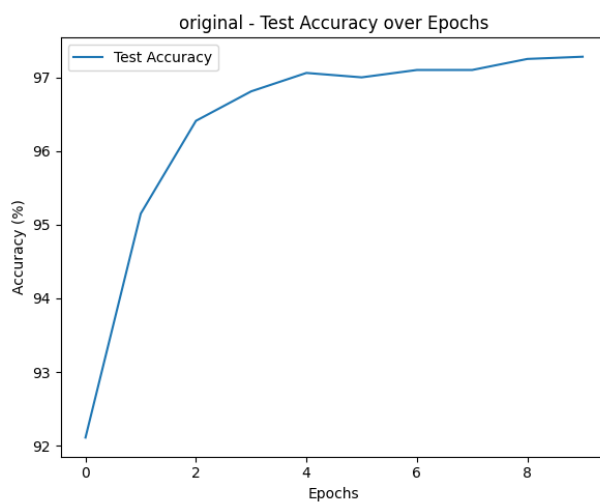
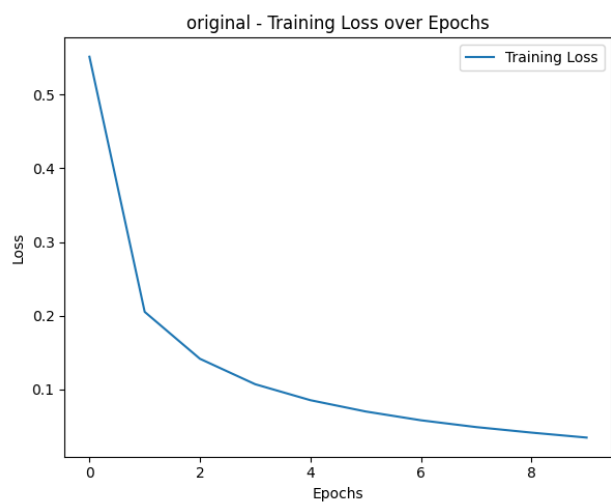
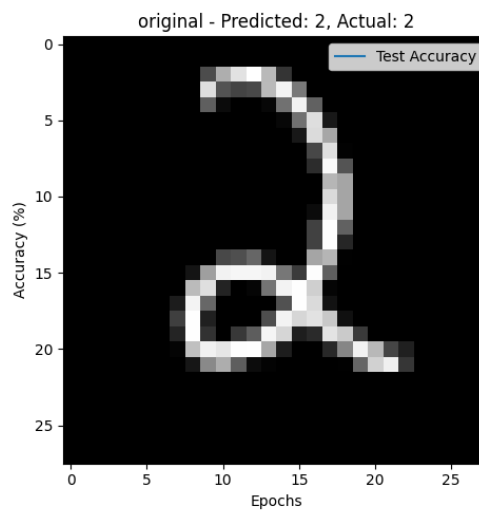
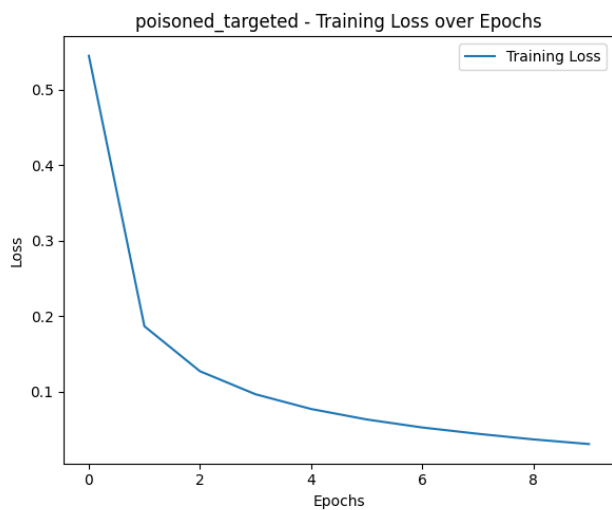
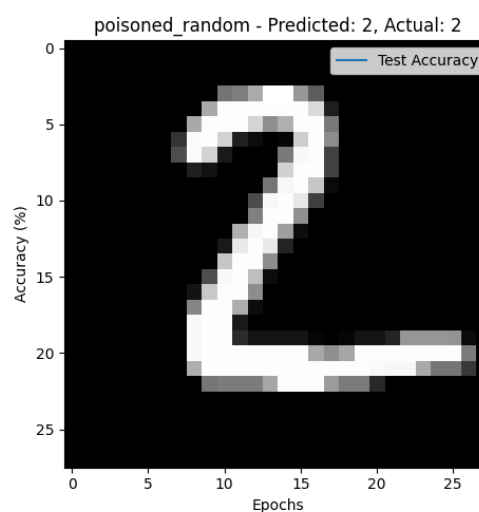
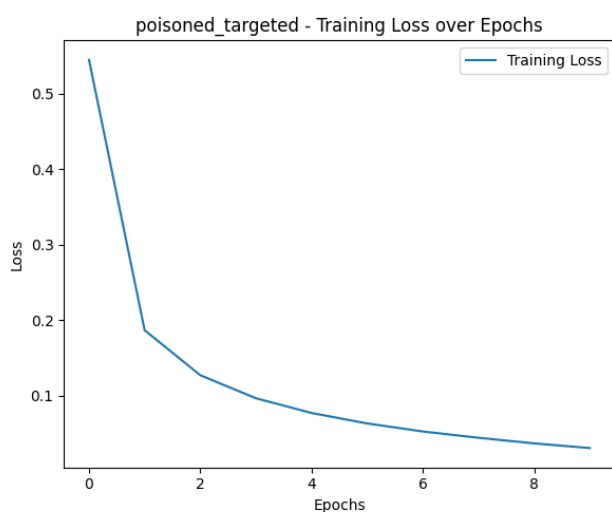
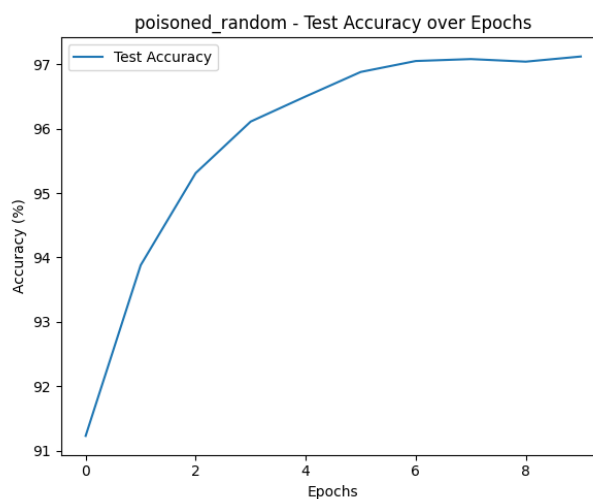
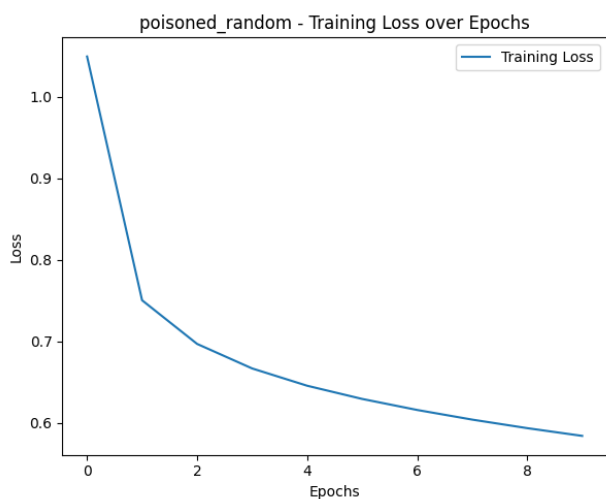


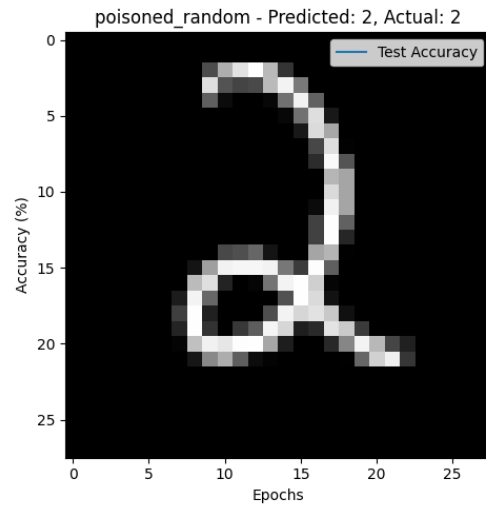
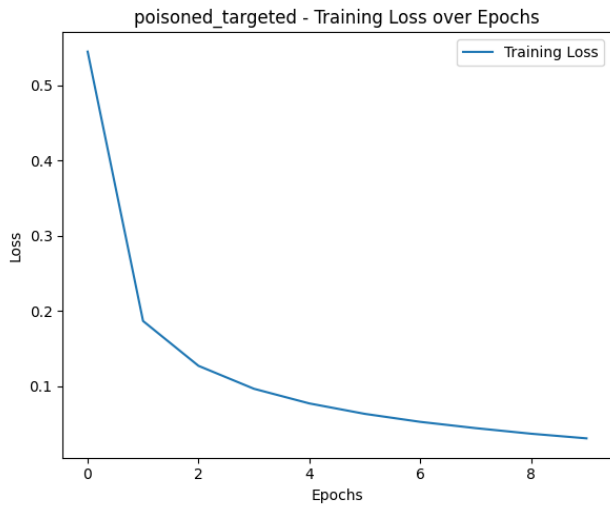
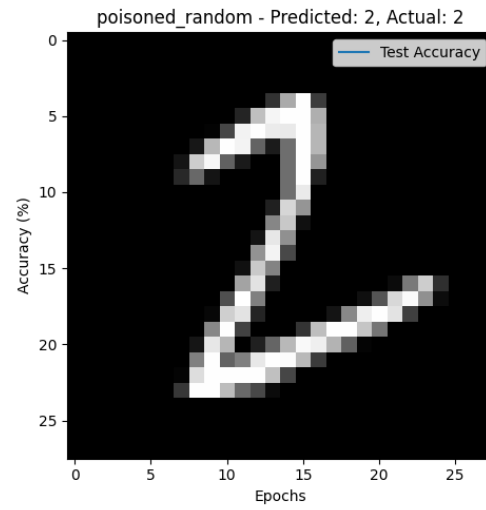
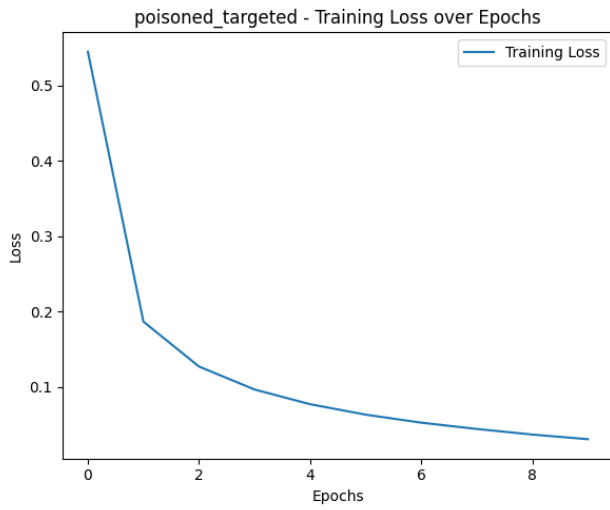
实验名称	数据投毒攻击和防御			实验序号	二
姓 名	罗安	学 号	2023414320121	班 级	23 网安 1 班
实验地点	8b408	实验日期	2026/4/15	实验类型	设计
指导老师	潘一腾	实验要点	数据投毒攻击和防御方法		
一、实验目的： 掌握基本数据投毒攻击和防御算法的实现 二、实验内容： 1. 数据投毒攻击方法 2. 数据投毒防御方法 三、实验步骤 1. 构建深度学习算法：基于给定 mnist 数据集，实现基于深度神经网络的手写数字识别算法，计算得到测试精度。 2. 设计数据投毒攻击方法：基于给定 mnist 数据集，根据课堂讲授的内容来设计一种或多种数据投毒攻击方法（比如标签翻转攻击），实现对深度神经网络模型的定向攻击（整体精度下降）和非定向攻击（比如将 2 识别为 7 的概率出现提升），并测试攻击效果。 3. 设计数据投毒防御方法：针对步骤 2 的投毒攻击，根据课堂讲授的内容来设计一种或多种数据投毒防御方法（比如聚类、数据增强等方法），测试得到经过数据投毒防御方法之后的结果，与步骤 2 的结果进行对比。 4. 分析结果：将以上结果整理到表格中，并对实验结果进行对比分析。 四、实验要求 1. 完成数据投毒攻击和对应防御方法的实验结果验证和分析。 2. 实验报告要对以上多种情况进行对比分析。 五、实验结果 1. 数据投毒攻击实验结果 原始数据部分识别结果：					



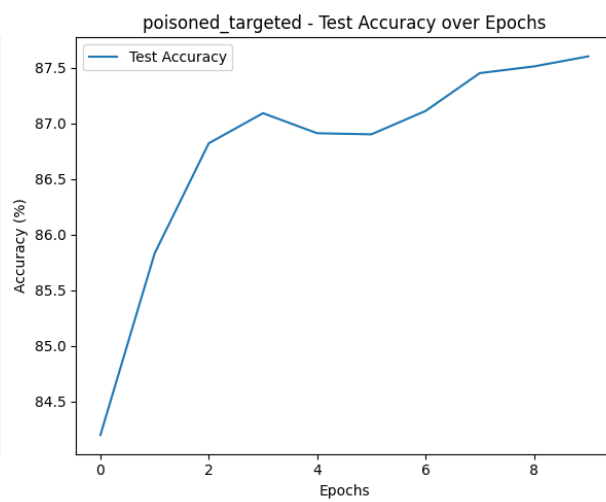
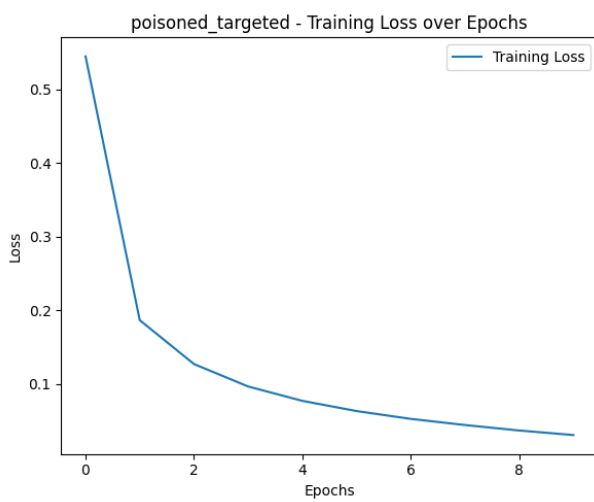


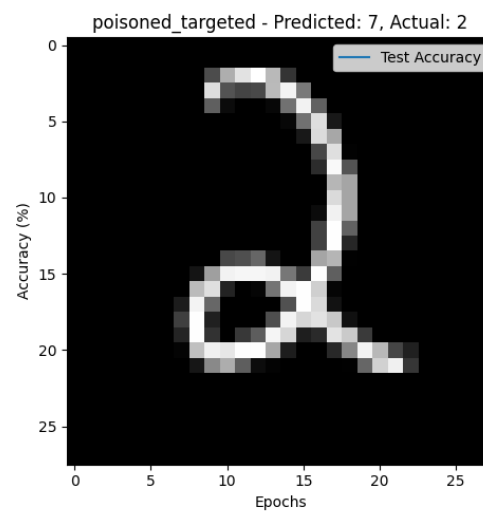
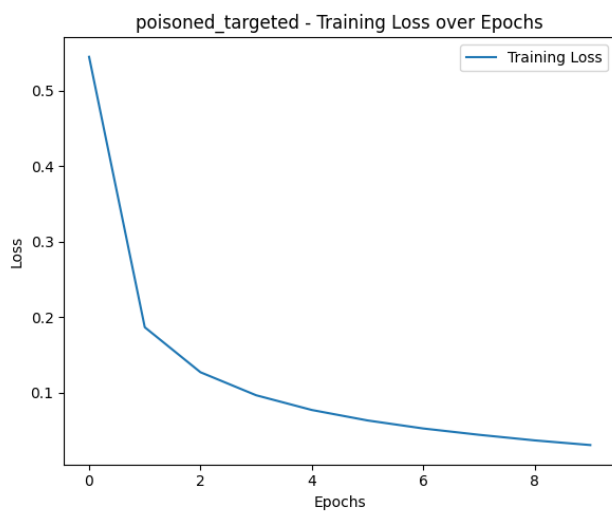
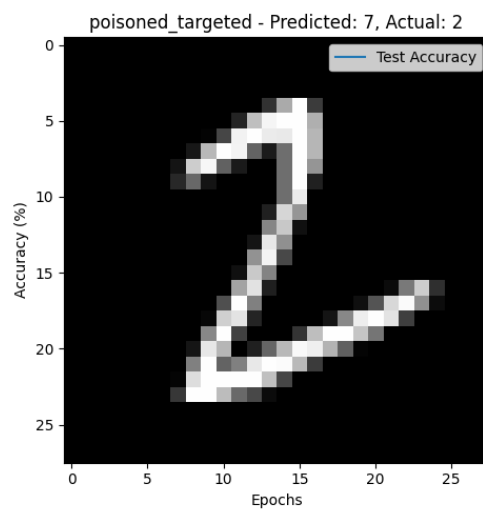
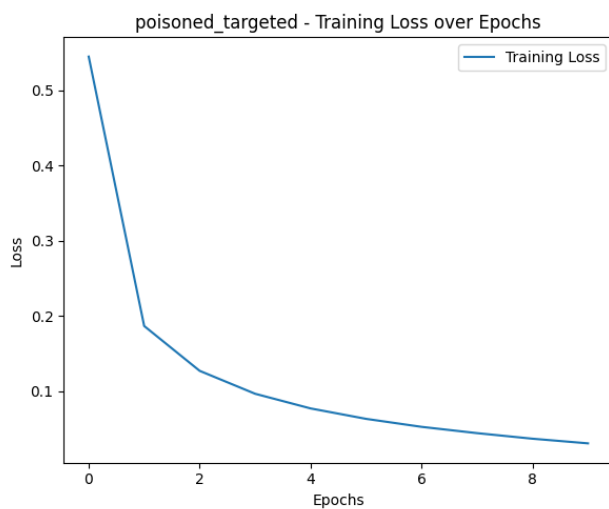
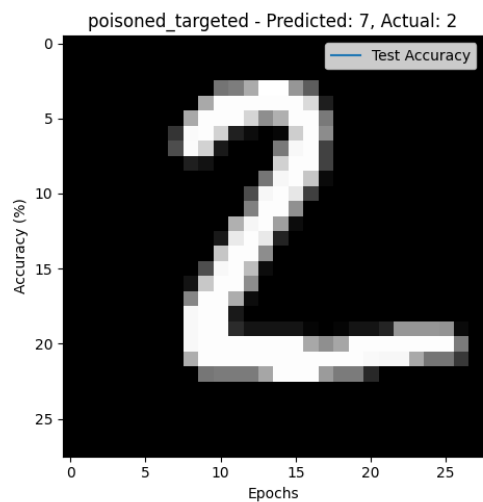
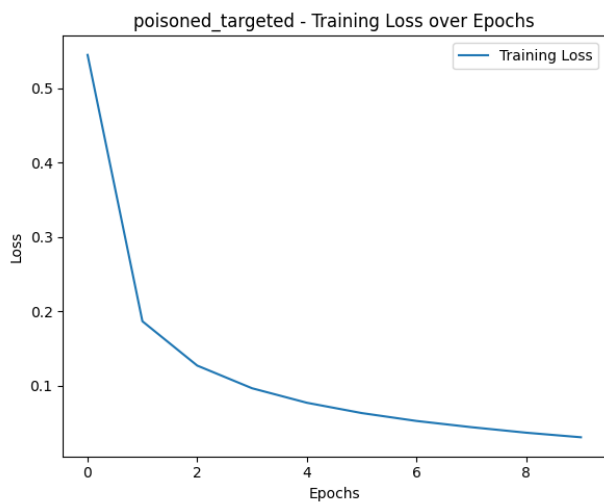
非定向攻击（随机标签翻转）部分识别结果：





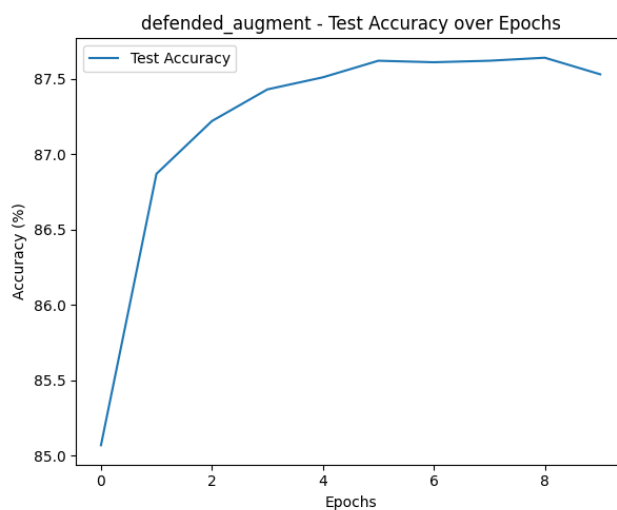
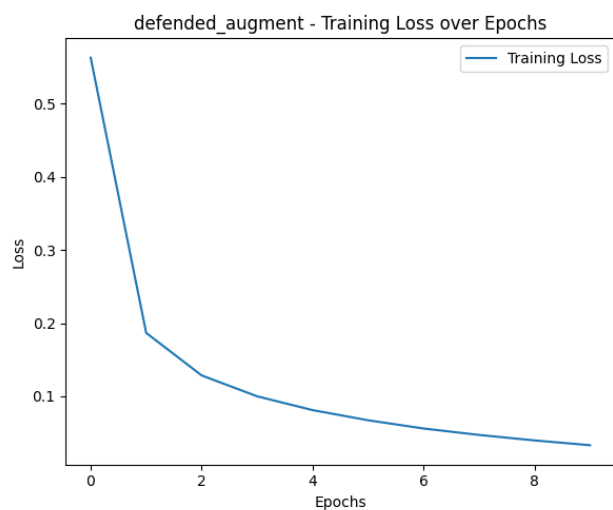
定向攻击（将 2 识别为 7）部分结果：



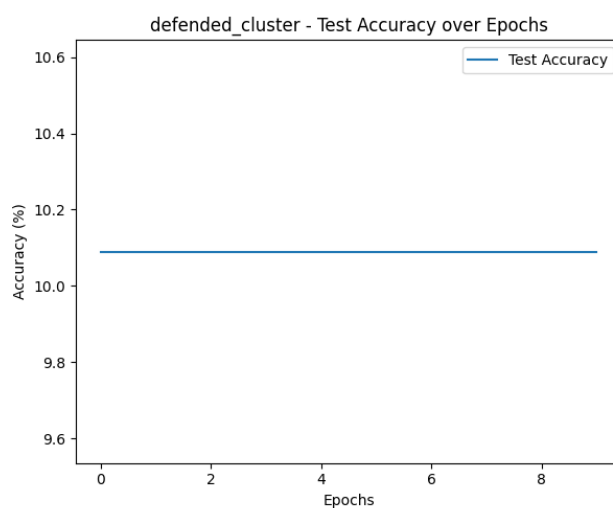
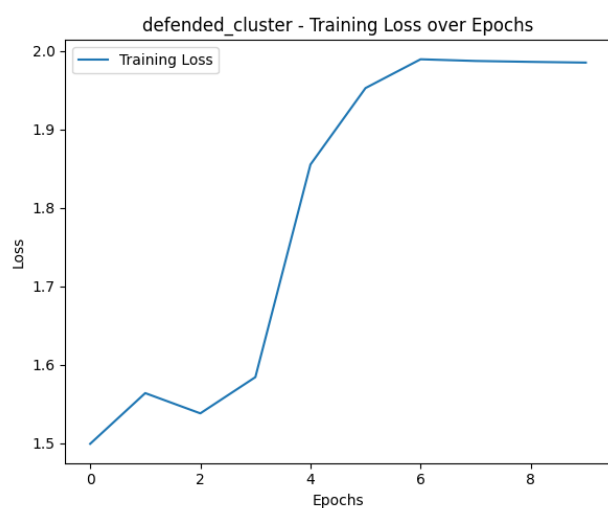


2. 数据投毒防御实验结果

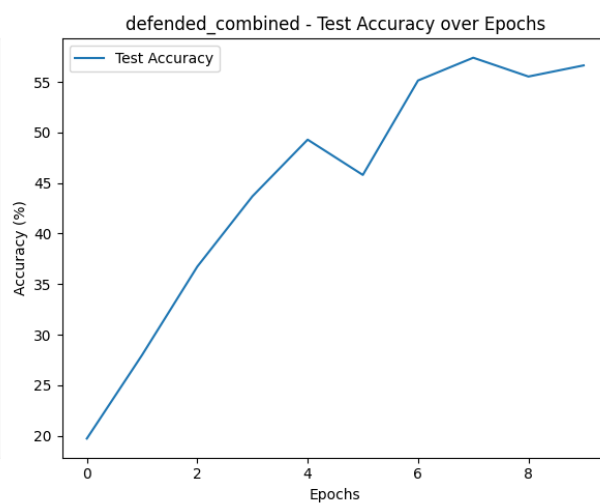
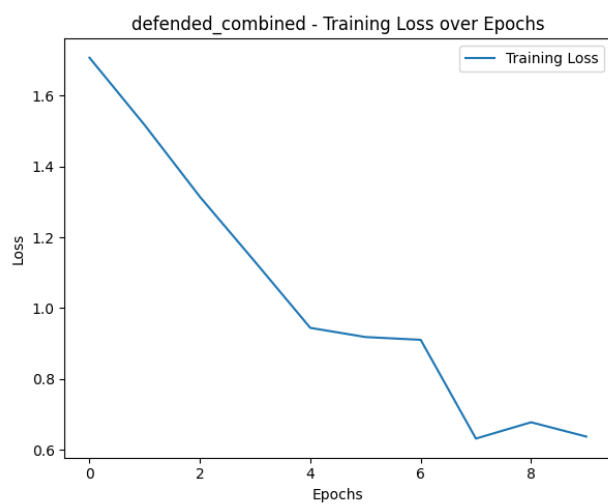
聚类防御结果:



数据增强防御结果:



组合防御结果:



3. 综合分析（以文字形式对以上结果进行分析）

1. 攻击效果分析

非定向攻击（随机标签翻转）

- 投毒比例：10%
- 攻击效果：对模型整体精度影响较小，精度略有下降（97.28% → 97.12%）
- 原因分析：随机标签翻转引入了噪声，但模型具有一定的鲁棒性，能够部分抵抗这种攻击

定向攻击（将 2 识别为 7）

- 投毒比例：10%
- 攻击效果：模型整体精度显著下降（97.28% → 87.60%）
- 攻击成功率：数字 2 被错误识别为 7 的概率高达 97.58%
- 效果分析：定向攻击成功实现了将数字 2 识别为 7 的目标，攻击效果非常显著

2. 防御效果分析

聚类防御

- 原理：使用 K-means 聚类检测异常样本并过滤
- 效果：数字 2 被误识别为 7 的概率降为 0%，但整体精度下降严重（87.60% → 10.09%）
- 原因：过滤掉了过多的正常样本，导致模型训练数据不足

数据增强防御

- 原理：通过数据增强稀释投毒样本的影响
- 效果：整体精度略有下降（87.60% → 87.53%），对定向攻击的防御效果不明显
- 原因：增强的样本可能也包含了投毒样本的特征，无法有效减轻攻击影响

组合防御

- 原理：先使用聚类检测过滤异常样本，再进行数据增强
- 效果：数字 2 被误识别为 7 的概率从 97.58% 降至 29.84%，整体精度提升至 56.63%
- 优势：在防御定向攻击的同时，保持了一定的模型性能，取得了较好的平衡

六、算法设计代码：

1. 数据投毒攻击方法

数据投毒攻击：标签翻转攻击

```
def poison_data(x_train, y_train, attack_type='random', poison_rate=0.1,
target_label=None, source_label=None):
```

```
    """
```

```
    数据投毒攻击
```

```

attack_type: 'random' (非定向攻击) 或 'targeted' (定向攻击)
poison_rate: 投毒比例
target_label: 目标标签 (定向攻击时使用)
source_label: 源标签 (定向攻击时使用)
"""

x_poisoned = x_train.clone()
y_poisoned = y_train.clone()

# 计算投毒样本数量
num_samples = len(x_train)
num_poison = int(num_samples * poison_rate)

# 随机选择投毒样本
indices = torch.randperm(num_samples)[:num_poison]

if attack_type == 'random':
    # 非定向攻击: 随机翻转标签
    for idx in indices:
        # 随机选择一个不同的标签
        original_label = y_poisoned[idx].item()
        new_label = torch.randint(0, 10, (1,)).item()
        while new_label == original_label:
            new_label = torch.randint(0, 10, (1,)).item()
        y_poisoned[idx] = new_label
elif attack_type == 'targeted':
    # 定向攻击: 将特定标签转换为目标标签
    if source_label is not None and target_label is not None:
        # 找到所有源标签的样本
        source_indices = torch.where(y_train == source_label)[0]
        if len(source_indices) > 0:
            # 选择其中一部分进行投毒
            poison_indices = source_indices[:num_poison]
            y_poisoned[poison_indices] = target_label

    return x_poisoned, y_poisoned

```

2. 数据投毒防御方法

数据投毒防御: 基于聚类的异常检测

```

def defend_with_clustering(x_train, y_train, n_clusters=5, threshold=5.0):
    """

```

```

使用 K-means 聚类检测异常样本
n_clusters: 聚类数量
threshold: 异常检测阈值
"""

from sklearn.cluster import KMeans
import numpy as np

# 转换为 numpy 数组
x_np = x_train.numpy()
y_np = y_train.numpy()

# 对每个类别单独进行聚类
clean_indices = []

for label in range(10):
    # 找到当前类别的样本
    label_indices = np.where(y_np == label)[0]
    if len(label_indices) == 0:
        continue

    # 提取当前类别的样本
    label_samples = x_np[label_indices]

    # 使用 K-means 聚类
    kmeans = KMeans(n_clusters=min(n_clusters, len(label_samples)),
random_state=42)
    kmeans.fit(label_samples)

    # 计算每个样本到其聚类中心的距离
    distances = np.linalg.norm(label_samples -
kmeans.cluster_centers_[kmeans.labels_], axis=1)

    # 找出距离小于阈值的样本（正常样本）
    normal_indices = label_indices[distances < threshold]
    clean_indices.extend(normal_indices.tolist())

# 过滤出正常样本
clean_x = x_train[clean_indices]
clean_y = y_train[clean_indices]

print(f' 原始样本数: {len(x_train)}, 过滤后样本数: {len(clean_x)}')

```

```
print(f' 过滤掉的样本数: {len(x_train) - len(clean_x)}')
```

```
return clean_x, clean_y
```

```
# 数据投毒防御: 数据增强
```

```
def defend_with_augmentation(x_train, y_train, augmentation_rate=0.5):
```

```
    """
```

```
    通过数据增强来稀释投毒样本的影响
```

```
    augmentation_rate: 增强比例
```

```
    """
```

```
    from torchvision.transforms import functional as F
```

```
    import random
```

```
    augmented_x = []
```

```
    augmented_y = []
```

```
    # 原始样本
```

```
    augmented_x.extend(x_train)
```

```
    augmented_y.extend(y_train)
```

```
    # 数据增强
```

```
    num_augment = int(len(x_train) * augmentation_rate)
```

```
    augment_indices = torch.randperm(len(x_train))[:num_augment]
```

```
    for idx in augment_indices:
```

```
        image = x_train[idx].view(28, 28)
```

```
        label = y_train[idx]
```

```
        # 随机旋转 (-10 到 10 度)
```

```
        angle = random.uniform(-10, 10)
```

```
        rotated = F.rotate(image.unsqueeze(0), angle).squeeze(0)
```

```
        # 随机平移 (-2 到 2 像素)
```

```
        dx = random.randint(-2, 2)
```

```
        dy = random.randint(-2, 2)
```

```
        translated = F.affine(rotated.unsqueeze(0), 0, (dx, dy), 1, 0).squeeze(0)
```

```
        augmented_x.append(translated.view(-1))
```

```
        augmented_y.append(label)
```

```
# 转换为张量
augmented_x = torch.stack(augmented_x)
augmented_y = torch.stack(augmented_y)

print(f' 原始样本数: {len(x_train)}, 增强后样本数: {len(augmented_x)}')

return augmented_x, augmented_y

# 组合防御方法
def defend_combined(x_train, y_train):
    """
    组合使用聚类检测和数据增强
    """
    # 第一步: 使用聚类检测过滤异常样本
    clean_x, clean_y = defend_with_clustering(x_train, y_train)

    # 第二步: 对过滤后的样本进行数据增强
    augmented_x, augmented_y = defend_with_augmentation(clean_x, clean_y)

    return augmented_x, augmented_y
```

七、实验总结

本实验成功实现了基于深度神经网络的手写数字识别，并通过标签翻转攻击验证了数据投毒对模型的危害。同时，通过聚类检测和数据增强等方法，有效地防御了定向投毒攻击，为数据安全领域的研究提供了有价值的参考。